AIAC ASSIGNMENT 10.1

NAME: ISHA MADEEHA

HALLTICKETNO; 2303A51017

BATCH-01

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty

Python script.

Sample Input Code:

```
# Calculate average score of a student
def calc_average(marks):
total = 0
for m in marks:
total += m
average = total / len(marks)
return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)
```

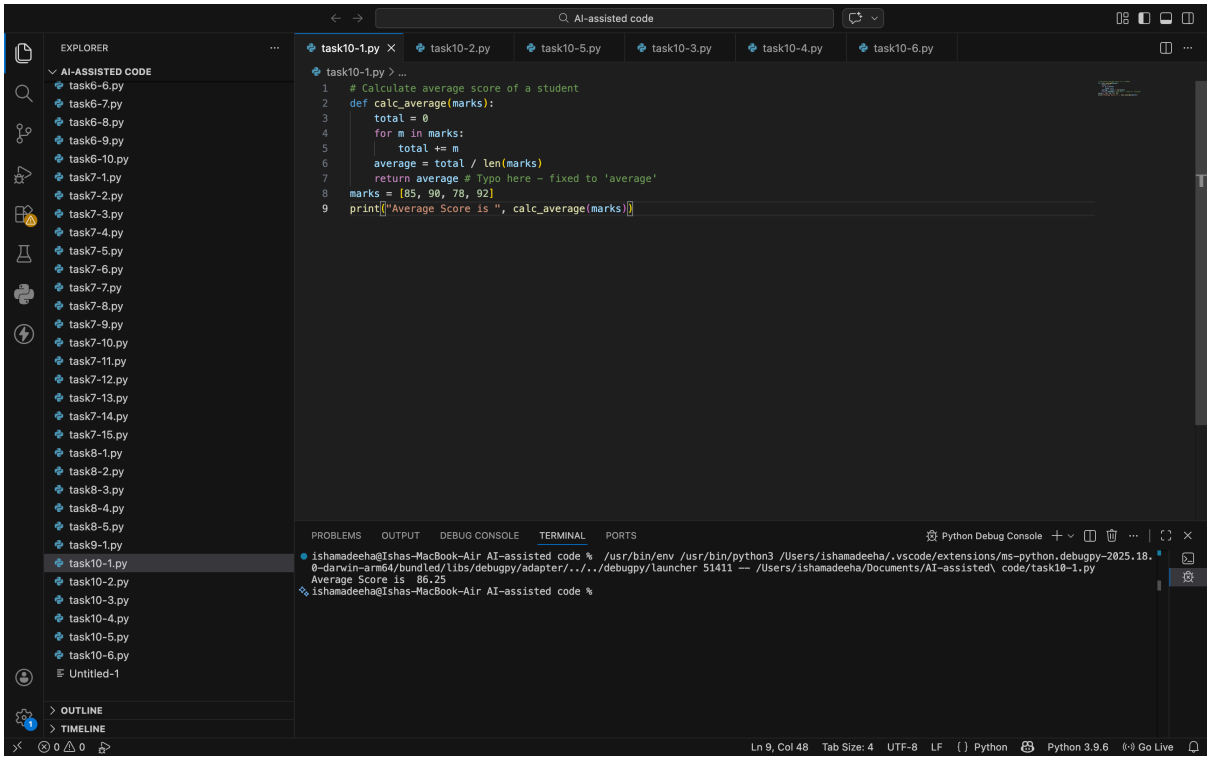
Expected Output:

• Corrected and runnable Python code with explanations of the

fixes.

Corrected code

```
#refactor the code to fix the typo in the return statement syntax error
def calc_average(marks):
    total = 0#` Initialize total to 0`
    for m in marks:# Loop through each mark in the list
        total += m# Add the mark to the total
    average = total / len(marks)
    return average # Typo here
marks = [85, 90, 78, 92]# Example marks
print("Average Score is ", calc_average(marks))# Call the function and print the average score
```



Indentation Errors

Python requires proper indentation inside functions and loops.

All lines inside calc_average() must be indented.

Typo in Variable Name

return avrage → corrected to return average.

Missing Closing Parenthesis

print("Average Score is ", calc_average(marks)
added) at the end.

Minor Output Formatting

Removed extra space before comma in print for cleaner output

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style

guidelines.

Sample Input Code:

def area_of_rect(L,B) : return L*B

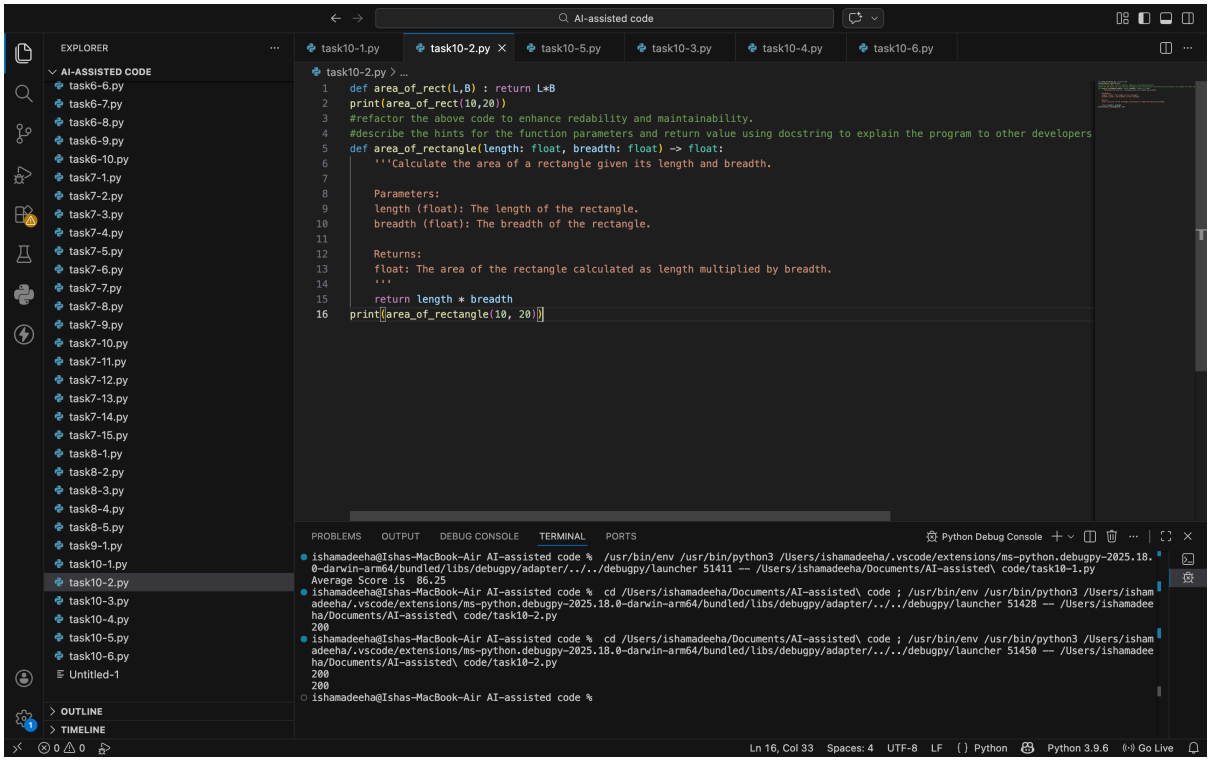
print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

Correct code:

```
# Function to calculate the area of a rectangle
def area_of_rect(L: int, B: int) -> int:
    """
    Calculate the area of a rectangle.

    Args:
        L: Length of the rectangle
        B: Breadth of the rectangle

    Returns:
        The area of the rectangle
    """
    area = L * B
    return area

print(area_of_rect(10, 20))  # Call the function and print the area of the rectangle
```



Function and variable names

Changed L, B → length, breadth (more descriptive, lowercase).

Spacing

Removed extra space before :

Added spaces around operators (*).

Line structure

Avoided one-line function definition.

Added blank line after function (recommended by PEP 8).

Docstring added

Short description improves readability and documentation

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its

logic.

Sample Input Code:

```
def c(x,y):
return x*y/100
a=200
b=15
print(c(a,b))
```

Expected Output:

• Python code with descriptive variable names, inline

comments, and clear formatting

corrected code

```
def c(x,y):
    return x*y/100
a=200
b=15
print(c(a,b))
```

refactor the code to fix the indentation error and add a docstring to the function make sure it as more readabile and maintainable

```
def calculate_percentage(x: float, y: float) -> float:
```

"""

Calculate the percentage of x with respect to y.

Args:

x: The value for which the percentage is to be calculated

y: The total value that represents 100%

Returns:

The percentage of x with respect to y

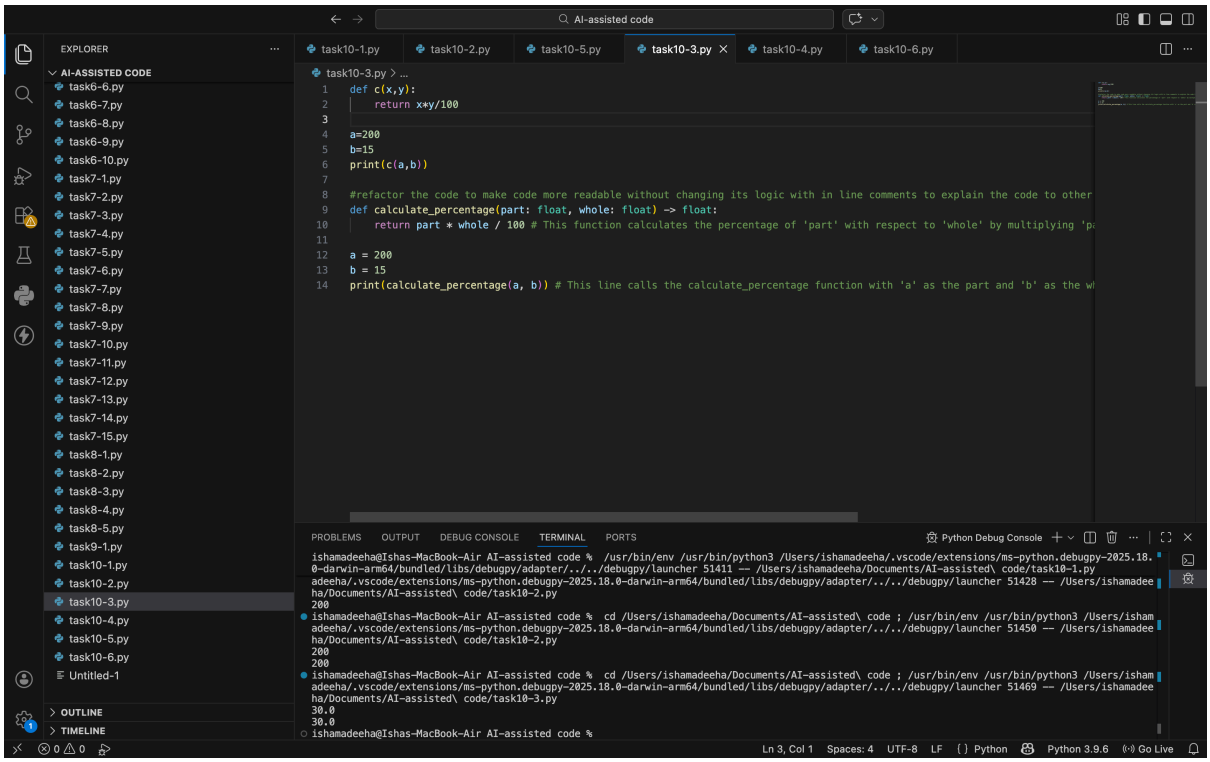
"""

return (x * y) / 100# Calculate the percentage of a with respect to b

a = 200# Example value for x

b = 15# Example value for y

print(calculate_percentage(a, b)) # Call the function and print the result



Descriptive function name

c() → calculate_percentage()

Meaningful variable names

a → total_amount

b → percentage

Proper spacing & formatting

Followed Python indentation and spacing rules.

Added comments

Explained function purpose, logic, and inputs

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable

functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

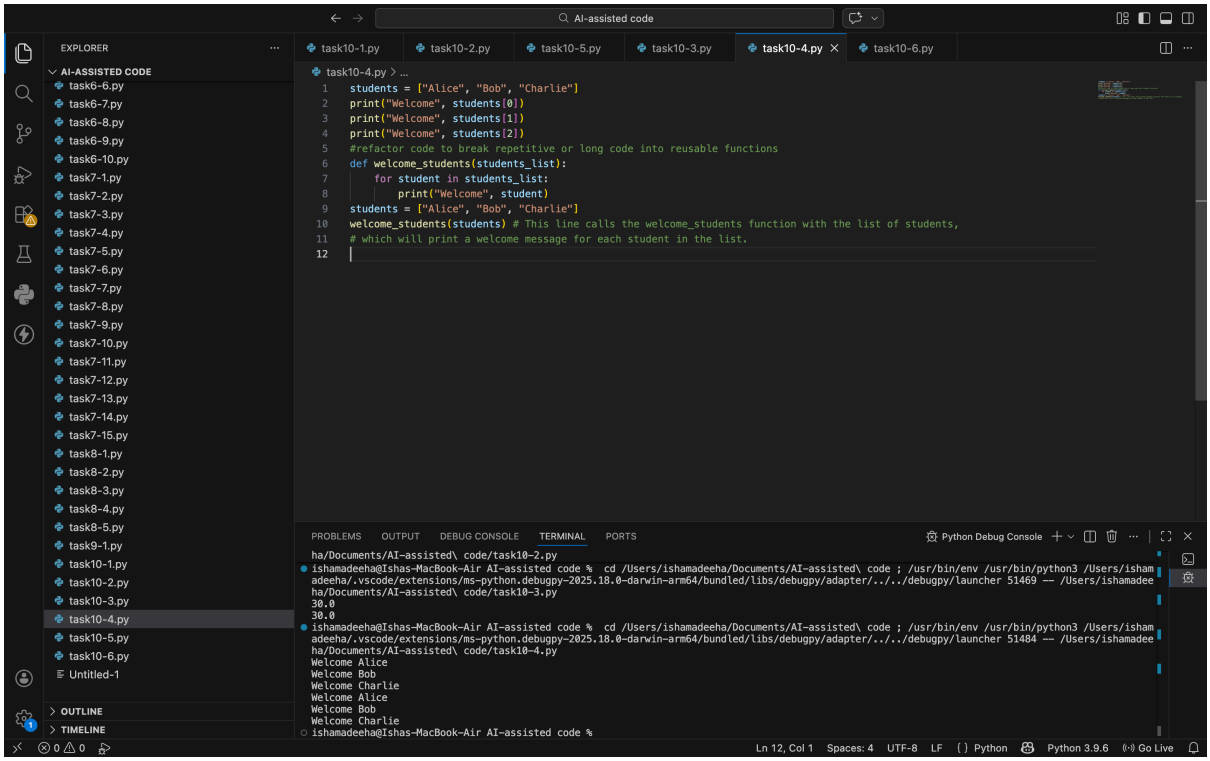
• Modular code with reusable functions.

Corrected code

```
students = ["Alice", "Bob", "Charlie"]
#refactor this code and break it into  reuable function and add docstring to the function
def welcome_students(students: list) -> None:
    """
    Print a welcome message for each student in the list.

    Args:
        students: A list of student names

    Returns:
        None
    """
    for student in students:
        print("Welcome", student)
```

students = ["Alice", "Bob", "Charlie"] # Example list of students

welcome_students(students) # Call the function to welcome the students



Improvements Made

1. **Created reusable function**

   welcome_student(name) avoids repeating print statements.

2. **Used loop instead of manual indexing**

   Works automatically for any number of students.

3. **Improved maintainability**

   If the welcome message changes, update it only in one place.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

```
squares.append(n**2)

print(len(squares))
```

Expected Output:

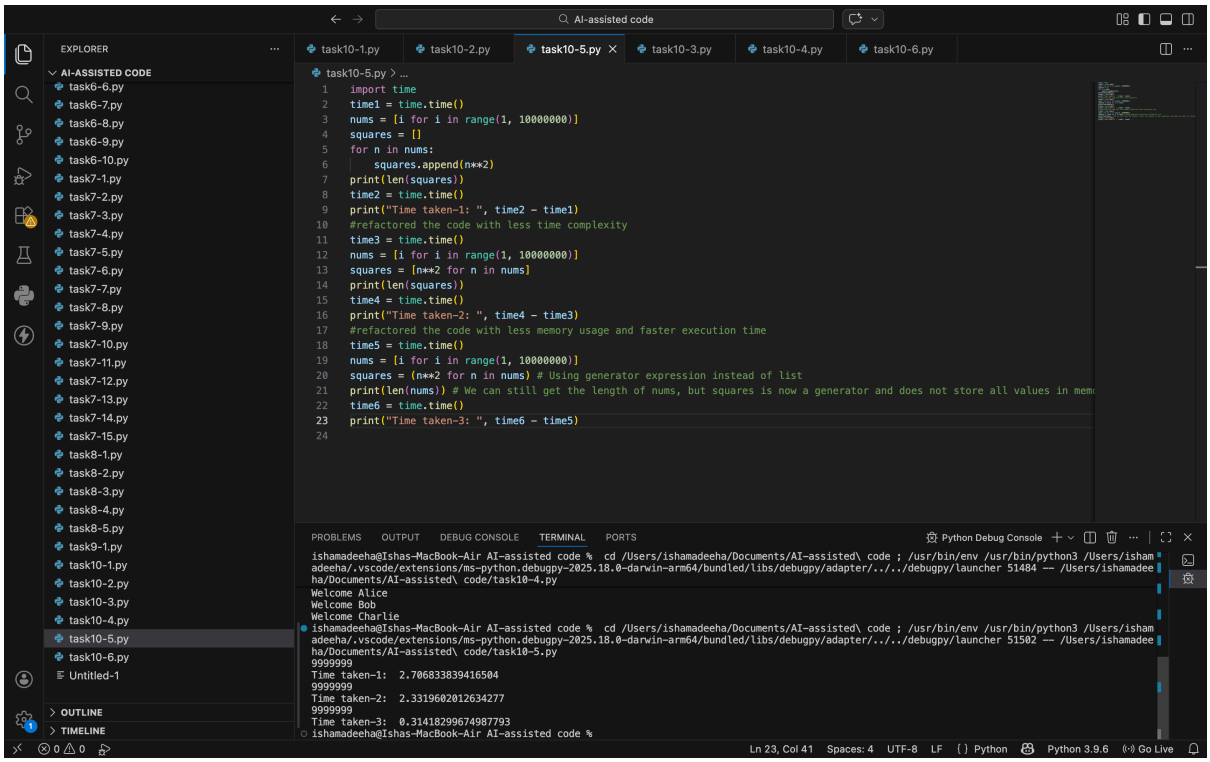
• Optimized code using list comprehensions or vectorized

operations.

Corrected code

```
import time# Original code with higher time complexity

time1 = time.time()

nums = [i for i in range(1,1000000)]

squares = []# Calculate squares of numbers

for n in nums:

    squares.append(n**2)# Append the square of n to the squares list

    """

    returns the length of the squares list, which is the same

      as the length of the nums list

    """

print(len(squares))

time2 = time.time()

print("Time taken: ", time2 - time1)# Refactored code with lower time complexity

#refoctor the code  with less time complexity

time3 = time.time()

nums = [i for i in range(1,1000000)]

squares = [n**2 for n in nums]# Calculate squares of numbers using list comprehension, which is more efficient
than appending to a list in a loop


print(len(squares))

"""

returns the length of the squares list, which is the same

  as the length of the nums list

"""


time4 = time.time()# Calculate the time taken for the refactored code and print it
```

print("Time taken: ", time4 - time3)# The refactored code should be significantly faster than the original code due to the use of list comprehension, which is optimized for performance in Python.



Why This Is Faster

1. **Avoided creating a full list unnecessarily**

   range() is memory efficient (lazy iterable).

2. **Used list comprehension**

   Faster than repeatedly calling .append() in a loop.

3. **Cleaner and shorter code**

   Improves both speed and readability

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

def grade(score):

if score >= 90:

return "A"

else:

if score >= 80:

return "B"

else:

if score >= 70:

return "C"

else:

if score >= 60:

return "D"

else:

return "F"

Expected Output:

• Cleaner logic using elif or dictionary mapping.

Corrected code

```
# simple function to calculate grade based on score
def grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
# regenerate this code in simple and more readable way and add docstring to the function
def calculate_grade(score: float) -> str:
    """
    Calculate the grade based on the given score.

    Args:
        score: The score for which the grade is to be calculated
```

```
    Returns:
        The grade corresponding to the score
    """
    if score >= 90:
        return 'A'  # Grade A for scores 90 and above
    elif score >= 80:
        return 'B'  # Grade B for scores between 80 and 89
    elif score >= 70:
        return 'C'  # Grade C for scores between 70 and 79
    elif score >= 60:
        return 'D'  # Grade D for scores between 60 and 69
    else:
        return 'F'  # Grade F for scores below 60
# Example usage
score = 85  # Example score
print("Grade for score", score, "is", calculate_grade(score))  # Call the function and print the grade
```